Practical Model Report: Amazon Reviews Sentiment and Aspect Analysis

Scope and evidence

This report is based on the following two SageMaker Studio notebooks:

1. `sagemaker/train_model_studio.ipynb` — review-level binary sentiment classification.
2. `sagemaker/aspect_model_studio.ipynb` — sentence-level, multi-label aspect detection with VADER sentiment.

Important limitation: both notebook files currently contain code but no saved execution outputs. Therefore, the repository does **not** provide numeric accuracy, F1, precision, recall, confusion-matrix, or Hamming-loss results. The notebooks calculate these values when run, but reporting invented percentages would be incorrect.

Model overview

Item	Model 1: Sentiment Backbone	Model 2: Aspect-Based Analysis
Main purpose	Classify a complete review as positive or negative	Detect the food-related aspects mentioned in each sentence and attach sentiment
Learning task	Binary classification	Multi-label classification plus lexicon-based sentiment
Input	Cleaned full-review text from `gold/model_input/`	Review sentences from `gold/aspect_sentences/`
Labels	Rating 1–2 = negative; rating 4–5 = positive; rating 3 excluded	Weak aspect labels generated from seed keywords
Text features	TF-IDF unigrams and bigrams, maximum 100,000 features	TF-IDF unigrams and bigrams, maximum 80,000 features
Algorithms	Logistic Regression, Linear SVM, Multinomial Naive Bayes	One-vs-Rest Logistic Regression: one classifier for each of nine aspects
Selection/evaluation	Best validation F1, then one evaluation on untouched test data	Validation/test weak-label agreement using micro-F1, macro-F1, per-aspect scores, and Hamming loss
Sentiment component	Selected binary classifier	VADER sentence polarity: positive, neutral, or negative
Output	Review sentiment and probability/decision score	Aspect names, confidence, sentiment, and sentiment score

Twenty practical questions and answers

1. What are the two models in this project?

The first is a **review-level binary sentiment model** that predicts whether an Amazon food review is positive or negative. The second is an **aspect-based analysis model** that identifies which subjects are discussed—such as taste, freshness, packaging, or delivery—and assigns sentence sentiment using VADER. Together they answer both “Did the customer like the product?” and “What exactly did the customer like or dislike?”

2. How was the training data prepared for Model 1?

Model 1 reads the Gold `model_input` Parquet data. It removes rows with missing IDs, text, or labels; accepts only labels 0 and 1; checks for duplicate IDs and duplicate text; and verifies that review IDs do not overlap between train, validation, and test sets. Ratings 1–2 become negative, ratings 4–5 become positive, and 3-star reviews are excluded from this binary task.

3. Why are 3-star reviews removed from Model 1?

Three-star reviews are ambiguous and are treated as neutral in the data pipeline. Excluding them creates a clearer binary classification problem. The trade-off is that the deployed model cannot represent neutral sentiment, so a three-class model would be needed if neutral predictions are required.

4. How is class imbalance handled in Model 1?

Only the training set is downsampled so that positive and negative classes have equal counts, with a maximum of 75,000 rows per class. Validation and test sets keep their original distributions. This lets the model learn both classes without evaluating it on an artificially balanced test set.

5. How is text converted into numeric features?

Both notebooks use TF-IDF. It gives higher importance to terms that are useful in a document but less common across the corpus. Unigrams and bigrams are included, so the models can learn individual words such as “stale” and short phrases such as “not good.” Sublinear term frequency reduces the influence of repeated words.

6. How does the project prevent data leakage?

TF-IDF is fitted only on training text and is then used to transform validation and test text. Model 1 validates that record IDs and cleaned review text are not duplicated across the data. Model 2 assigns every sentence from the same review to one split, preventing sentences from one review appearing in both training and evaluation data.

7. Which candidate algorithms are tested for Model 1?

The notebook compares Logistic Regression, Linear Support Vector Machine (`LinearSVC`), and Multinomial Naive Bayes. All three are suitable classical baselines for sparse, high-dimensional TF-IDF text features.

8. Which is the best Model 1 algorithm?

The code defines the best algorithm as the one with the highest **validation F1 score**, using average precision as the tie-breaker. The notebook has no saved results, so it is not possible to truthfully name Logistic Regression, Linear SVM, or Naive Bayes as the actual winner yet. After execution, the first row of `validation_results_df` and the printed `Selected model:` line provide the answer.

9. Why is F1 used to choose the best sentiment model instead of accuracy alone?

F1 combines precision and recall. Accuracy can look high on an imbalanced dataset even when the minority class is poorly detected. Because the validation and test sets keep the real class distribution, F1 gives a more useful balance between missed positive reviews and incorrect positive predictions.

10. What metrics are calculated for Model 1?

The notebook calculates accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, and average precision for every validation candidate. For the selected model it also calculates the same test metrics, a classification report, and a confusion matrix. This combination shows overall correctness, class balance, ranking quality, and the types of mistakes made.

11. What is Model 1's accuracy?

The numeric accuracy is **not available in the committed notebook**, because its output cells are empty. Running the notebook will display validation accuracy for all three candidates and test accuracy for the selected candidate. The test value should be quoted as final performance only after confirming the run ID, dataset version, split sizes, and confusion matrix.

12. How does Model 1 make a prediction for a new review?

The saved pipeline first applies the fitted TF-IDF transformation and then calls the selected classifier. Logistic Regression or Naive Bayes returns a positive probability; Linear SVM returns a decision score. A predicted label of 0 means negative and 1 means positive. Packaging both steps in one pipeline ensures that inference uses exactly the training vocabulary and IDF weights.

13. What practical business problem does Model 1 solve?

It can monitor overall customer satisfaction at scale, compare products or time periods, prioritize negative reviews for investigation, and provide an overall sentiment trend. It cannot explain the cause of the sentiment, which is why the aspect model is also needed.

14. How was Model 2 created?

Model 2 begins with a domain lexicon covering nine aspects: taste, freshness, texture, packaging, delivery, price/value, ingredients/health, portion/quantity, and quality. Keyword matches create weak multi-label targets. TF-IDF features are then used to train nine class-balanced Logistic Regression classifiers through a One-vs-Rest strategy.

15. Why is Model 2 called multi-label rather than multi-class?

A sentence can discuss more than one aspect at once. For example, “The chocolate tastes excellent but the package arrived damaged” contains both taste and packaging. Multi-class classification would force only one answer, while multi-label classification can activate both aspects.

16. What role does LDA play in Model 2?

Latent Dirichlet Allocation discovers twelve unsupervised topics from a sample of up to 60,000 sentences. The notebook compares the leading topic words with aspect seed terms. LDA is used to inspect themes, discover missing vocabulary, and refine the taxonomy; it is not used as supervised proof of aspect-model accuracy.

17. How are aspect predictions produced?

Each of the nine Logistic Regression classifiers produces a probability for its aspect. An aspect is returned when its probability is at least 0.50. If several probabilities cross the threshold, several aspects are returned. In the interactive example, if none crosses the threshold, the output explicitly says “no aspect above threshold” and shows the highest available confidence.

18. How is sentiment assigned to each detected aspect?

The notebook applies VADER to the sentence. A compound score of at least 0.05 is positive, at most −0.05 is negative, and values between those limits are neutral. That same sentence score is attached to every detected aspect in the sentence. This is a practical baseline, but it can fail when two aspects in one sentence have opposite sentiment.

19. What are Model 2’s accuracy and best model?

Ordinary accuracy is not the main metric because this is a multi-label task. The notebook reports validation/test micro-F1, macro-F1, per-aspect precision/recall/F1, and test Hamming loss. No numeric results are saved in the notebook. There is also no algorithm competition: the implemented baseline is One-vs-Rest Logistic Regression. Its reported values measure agreement with keyword-generated weak labels, **not human-labelled real-world accuracy**.

20. Which model is best overall, and how should the project be improved?

Neither model is universally better because they solve different tasks. Model 1 is best for fast overall positive/negative classification; Model 2 is best for actionable explanations such as “packaging complaints are negative.” The next evaluation step should run both notebooks, preserve their outputs, and create a manually annotated aspect test set. Model 2 should then be evaluated against human labels, its 0.50 thresholds tuned per aspect, and aspect-specific sentiment tested on sentences containing mixed opinions.

How to report the final measured results

After executing the notebooks, complete the following table from their displayed outputs and generated JSON metadata files:

Result	Value to copy
Best Model 1 candidate	First row of `validation_results_df` / `Selected model:`
Model 1 validation F1	`f1` for the selected validation row
Model 1 test accuracy	`test_metrics["accuracy"]`
Model 1 test precision	`test_metrics["precision"]`
Model 1 test recall	`test_metrics["recall"]`
Model 1 test F1	`test_metrics["f1"]`
Model 1 test ROC-AUC	`test_metrics["roc_auc"]`
Model 2 validation micro/macro-F1	`weak_metrics`
Model 2 test micro/macro-F1	`weak_metrics`
Model 2 Hamming loss	`weak_metrics["test_hamming_loss"]`
Strongest/weakest aspects	Highest/lowest rows of `per_aspect_df` by `f1-score`

Conclusion

The two notebooks form a complementary classical NLP solution. The sentiment backbone uses leakage-safe model comparison to summarize whole-review opinion. The aspect pipeline uses weak supervision and multi-label classification to explain the topics behind that opinion. The methodology is reproducible and practical, but the current repository cannot support claims such as “the model achieved 90% accuracy” until the notebooks are executed and their outputs are saved. In particular, the aspect model needs human-labelled evaluation before its weak-label scores can be described as genuine accuracy.